Velbaz — Les clés API à mettre dans Admin Panel → API Keys

Aucune clé n'est obligatoire. Velbaz démarre et fonctionne sans aucune d'elles. Chaque clé active **un outil précis** ; sans elle, l'outil est désactivé proprement (bouton masqué ou mode démo affiché comme tel, jamais de faux résultat).

Où les saisir : **Admin Panel → API Keys**. C'est chiffré en base et pris en compte tout de suite, sans redémarrage.

Les outils qui ont vraiment besoin d'une clé

Remplis seulement les outils que tu veux utiliser.

#	Outil (ce que ça débloque)	Clés à saisir	Où l'obtenir
1	**Stripe** : vrais paiements (plans, packs de crédits)	`STRIPE_SECRET_KEY` · `STRIPE_WEBHOOK_SECRET` (optionnel)	dashboard.stripe.com → Developers → API keys / Webhooks
2	**Resend** : envoi d'e-mails	`RESEND_API_KEY` · `RESEND_FROM`	resend.com → API Keys
3	**Printify** : vrais produits print-on-demand et commandes	`PRINTIFY_API_TOKEN` · `PRINTIFY_SHOP_ID` (optionnel)	Printify → Settings → Connections → API
4	**Higgsfield** : génération d'images et de vidéos	`HF_API_KEY` · `HF_API_SECRET`	higgsfield.ai → API
5	**Replicate** : upscale IA des visuels pour l'impression	`REPLICATE_API_TOKEN`	replicate.com → Account → API tokens
6	**21st.dev** : composants premium dans le builder	`TWENTY_FIRST_API_KEY`	21st.dev → Magic → API key
7	**Firecrawl** : scraping / clonage de sites existants	`FIRECRAWL_API_KEY`	firecrawl.dev → API Keys
8	**OpenRouter** : modèles « non filtrés » uniquement	`OPENROUTER_API_KEY`	openrouter.ai → Keys
9	**GitHub** : export des projets vers GitHub	`GITHUB_TOKEN` (**jeton classique `ghp_…`, case `repo` cochée**) · `GITHUB_OWNER`	github.com → Settings → Developer settings → Personal access tokens → **Tokens (classic)** → Generate new token (classic) → cocher `repo`. Un jeton « fine-grained » `github_pat_…` est refusé par GitHub (403) à la création du dépôt.
10	**Sentry** : surveillance des erreurs	`SENTRY_DSN` · `SENTRY_AUTH_TOKEN` · `SENTRY_ORG` · `SENTRY_PROJECT`	sentry.io → Settings
11	**Google** : bouton « Continuer avec Google »	`GOOGLE_CLIENT_ID` · `GOOGLE_CLIENT_SECRET`	console.cloud.google.com → Credentials → OAuth client ID
12	**Bland.ai** : vrais appels IA + SMS (Growth Engine)	`BLAND_API_KEY` · `BLAND_FROM` (obligatoire pour les SMS)	app.bland.ai → API / Phone Numbers
13	**Discord** : connexion et publication	`DISCORD_CLIENT_ID` · `DISCORD_CLIENT_SECRET` · `DISCORD_BOT_TOKEN`	discord.com/developers
14	**Instagram** : connexion et publication	`INSTAGRAM_CLIENT_ID` · `INSTAGRAM_CLIENT_SECRET`	developers.facebook.com
15	**Reddit** : connexion et publication	`REDDIT_CLIENT_ID` · `REDDIT_CLIENT_SECRET`	reddit.com/prefs/apps
16	**X / Twitter** : connexion et publication	`TWITTER_CLIENT_ID` · `TWITTER_CLIENT_SECRET`	developer.x.com
17	**Google Ads** : chaque client connecte SON compte Google Ads, l'IA y crée des campagnes **en pause**	`GOOGLE_ADS_DEVELOPER_TOKEN` · `GOOGLE_ADS_CLIENT_ID` + `GOOGLE_ADS_CLIENT_SECRET` (vides = ceux de la ligne 11) · `GOOGLE_ADS_LOGIN_CUSTOMER_ID` (optionnel, ton compte administrateur MCC)	Developer token : ads.google.com → compte administrateur (MCC) → Outils → Centre API. OAuth : console.cloud.google.com → activer « Google Ads API » → Credentials

#	Outil (ce que ça débloque)	Clés à saisir	Où l'obtenir
18	**Meta Ads** (Facebook / Instagram) : chaque client connecte SON compte publicitaire, l'IA y crée des campagnes **en pause**	`META_APP_ID` · `META_APP_SECRET`	developers.facebook.com → My Apps → Create app (type Business) → produits « Facebook Login for Business » + « Marketing API »
19	**Domaines** : un domaine client connecté affiche VRAIMENT son site (HTTPS compris)	`CLOUDFLARE_API_TOKEN` · `CLOUDFLARE_ZONE_ID` · `VELBAZ_DOMAIN_TARGET` · `VELBAZ_EDGE_SECRET`	dash.cloudflare.com — voir la section « Domaines » plus bas
20	**Namecheap** : achat de domaine depuis Velbaz + DNS écrits automatiquement	`NAMECHEAP_API_USER` · `NAMECHEAP_API_KEY` · `NAMECHEAP_USERNAME` · `NAMECHEAP_CLIENT_IP`	namecheap.com → Profile → Tools → API Access (activer, puis autoriser l'IP du serveur)

Google Ads et Meta Ads — ce qu'il faut savoir

- Où : onglet **Ads** d'un projet → bloc « Comptes publicitaires » → **Connecter** (fenêtre Google / Facebook). Le client choisit ensuite son compte (et sa Page Facebook pour Meta). Ensuite, chaque pub du projet a un bouton pour l'envoyer sur la régie.
- **Toutes les campagnes sont créées EN PAUSE**, budget plafonné à 50 / jour (10 par défaut). Rien ne dépense tant que le client ne l'active pas lui-même dans Google Ads / Meta.
- **URL de retour OAuth à déclarer** (remplace `<base>` par l'adresse de Velbaz, ex. `https://velbaz.com`) :
- Google (Credentials → ton client OAuth → Authorized redirect URIs) : `<base>/api/ad-accounts/callback/google_ads`
- Meta (Facebook Login for Business → Settings → Valid OAuth Redirect URIs) : `<base>/api/ad-accounts/callback/meta_ads`
- L'écran « Ads » affiche l'URL exacte à copier tant que les clés manquent.
- **Google** : un developer token neuf est en accès « Test » (comptes de test seulement). Pour de vrais comptes clients, demande l'accès **Basic** dans le Centre API (formulaire, quelques jours). L'écran de consentement OAuth doit inclure le scope `https://www.googleapis.com/auth/adwords`.
- **Meta** : sans App Review, seuls les admins / testeurs de l'app peuvent se connecter. Pour tous les clients : passer l'**App Review** pour `ads_management`, `ads_read`, `business_management`, `pages_show_list`, `pages_read_engagement`, avec la vérification d'entreprise (Business Verification).
- Pas encore testé contre de vrais comptes : les premiers essais diront si un réglage manque.

Sans Bland, appels et SMS du Growth Engine sont en **mode démo** (affiché comme tel, aucun envoi réel). Les SMS exigent un numéro Bland (`BLAND_FROM` , plan Agent Phone Plan ou Enterprise) et ne partent que vers les États-Unis et le Canada (max 100/jour).

Accès admin : `ADMIN_EMAILS` n'est pas obligatoire. Le code donne déjà l'accès admin à `johnemadmansour1@gmail.com` par défaut. Ajoute-le seulement pour donner l'accès à d'autres e-mails (séparés par des virgules, dans `.env` , puis redémarrage).

Ce que tu n'as PAS besoin de fournir

Déjà branché par Runable (16 clés déjà dans `.env` , n'y touche pas) : base de données, stockage de fichiers, authentification, passerelle IA.

IA (GPT, Claude, Gemini…) : aucune clé. Tous les agents passent par la passerelle IA Runable. Pas de `OPENAI_API_KEY` , `ANTHROPIC_API_KEY` ni `GEMINI_API_KEY` à mettre.

Hébergement : aucune clé. L'hébergement de Velbaz passe par Runable. Le jeton de déploiement n'est pas nécessaire. **Les domaines clients, eux, ont besoin des lignes 19 (et 20 pour l'achat)** : sans elles, un domaine peut être enregistré mais il n'affichera pas le site, et Velbaz le dit clairement au lieu de le marquer « en ligne ».

Ces outils marchent déjà sans clé : - recherche web (DuckDuckGo) - disponibilité des noms de domaine (RDAP) - vérification DNS des domaines - données crypto (Binance / CoinGecko publics) - photo de profil (Gravatar) - upscale local (sharp) - publication interne des sites (`/s/ton-site`) - indexation IndexNow

Demandées dans le chat, pas dans le panel : les clés propres à UNE entreprise (CJ Dropshipping `CJ_EMAIL` / `CJ_API_KEY` , clé d'exchange crypto, Printify ou Replicate propres à l'entreprise). Une fenêtre sécurisée les demande au moment où l'entreprise en a besoin.

Ce ne sont pas des clés (réglages techniques, déjà bons par défaut) : `PUBLIC_BASE_URL` , `APP_BASE_URL` , `VELBAZ_URL` , `VELBAZ_API_URL` , les noms de modèles (`AI_MODEL` , `TIER_MODEL_*` …), `SELFHEAL_DISABLED` , `X_LIVE_ENABLED` , `BETA_*` , `VELBAZ_WAITLIST` , `LAUNCH_*` , `*_REDIRECT_BASE` , `*_BASE_URL` .

Optionnel et très spécifique : `INGEST_API_KEY` + `VELBAZ_CONTROL_BASE_URL` servent seulement si tu fais tourner ton propre serveur de contrôle Velbaz (télémétrie). Sinon ignore-les. Le message `INGEST_API_KEY not set` dans les logs est sans conséquence.

Domaines clients : la mise en place (une seule fois)

Pourquoi : pour qu'un domaine client (ex. `maboutique.com`) affiche son site, il faut (1) que son DNS pointe vers une adresse qui existe, (2) un certificat HTTPS pour ce domaine, (3) que la requête arrive à Velbaz avec le nom du domaine d'origine. Velbaz ne peut pas faire ça seul : il faut **Cloudflare for SaaS + un petit Worker** sur un domaine à toi géré par Cloudflare. (`velbaz.com` est aujourd'hui chez Hostinger, pas chez Cloudflare.)

1. **Un domaine à toi sur Cloudflare** (ex. `velbaz-sites.com`, ou `velbaz.com` si tu le passes chez Cloudflare). Son « Zone ID » (page Overview) → `CLOUDFLARE_ZONE_ID`.
2. **Activer Cloudflare for SaaS** : SSL/TLS → Custom Hostnames (gratuit jusqu'à 100 domaines).
3. **Créer l'adresse cible** : DNS → enregistrement `sites` de type A vers `192.0.2.1`, **proxied (nuage orange)**. Puis dans Custom Hostnames → **Fallback Origin** = `sites.velbaz-sites.com`. Ce nom → `VELBAZ_DOMAIN_TARGET`.
4. **Créer le Worker** (Workers & Pages → Create → Hello World), coller le code ci-dessous, puis Settings → Variables : `VELBAZ_ORIGIN` = l'adresse publiée de Velbaz sans https (ex. `velbaz.com`), et le secret `VELBAZ_EDGE_SECRET` = une longue chaîne aléatoire (32+ caractères). Mets **la même chaîne** dans Velbaz (`VELBAZ_EDGE_SECRET`).
5. **Route du Worker** : sur la zone, Workers Routes → route `*/*` → ce Worker (elle capte aussi tous les domaines clients).
6. **Jeton API** : My Profile → API Tokens → Create Token → permissions **Zone · SSL and Certificates · Edit** (+ Zone · Zone · Read), limité à cette zone → `CLOUDFLARE_API_TOKEN`.
7. **Namecheap (achat + DNS auto)** : Profile → Tools → API Access → activer, et ajouter l'IP publique du serveur Velbaz à la liste blanche → `NAMECHEAP_CLIENT_IP`. Le compte doit avoir une adresse par défaut (Profile → Addresses) : elle sert de titulaire du domaine acheté, et le solde du compte paie l'achat.

```
export default {
  async fetch(request, env) {
    const url = new URL(request.url);
    const siteHost = url.hostname;          // domaine du client
    url.hostname = env.VELBAZ_ORIGIN;       // ex. velbaz.com
    const headers = new Headers(request.headers);
    headers.set('x-velbaz-site-host', siteHost);
    headers.set('x-velbaz-edge-key', env.VELBAZ_EDGE_SECRET);
    return fetch(url.toString(), {
      method: request.method,
      headers,
      body: ['GET', 'HEAD'].includes(request.method) ? undefined : request.body,
      redirect: 'manual',
    });
  },
};
```

Ce que fait Velbaz ensuite, tout seul : quand un client connecte un domaine, Velbaz crée le « custom hostname » Cloudflare (HTTPS), lui donne les DNS à créer (ou les écrit lui-même via Namecheap, bouton « Configurer le DNS automatiquement »), puis **appelle le domaine** (`/api/domains/whoami`). Le domaine ne passe « en ligne » que si c'est bien Velbaz qui répond avec le bon site. Tant que `VELBAZ_DOMAIN_TARGET` n'existe pas, l'onglet Domaines affiche un bandeau rouge qui le dit.

Non vérifié en vrai : aucun achat Namecheap ni aucune écriture DNS n'a été lancé sur un vrai compte.